

llama.cpp is an open-source software library that performs inference on various large language models such as Meta's Llama model. It is co-developed alongside the GGML project, a general-purpose tensor library.

Command-line tools are included with the library, alongside a server with a simple web interface.

llama.cpp has been considered as the de facto standard as the core of almost all local inference tools, including Ollama and LM Studio.

Background

Towards the end of September 2022, Georgi Gerganov started work on the GGML library, a C library implementing tensor algebra. Gerganov developed the library with the intention of strict memory management and multi-threading. The creation of GGML was inspired by Fabrice Bellard's work on LibNC.

Before llama.cpp, Gerganov worked on a similar library called whisper.cpp which implemented Whisper, a speech to text model by OpenAI.

Development

llama.cpp began development in March 2023 by Georgi Gerganov as an implementation of the Llama inference code in pure C/C++ with no dependencies. This improved performance on computers without GPU or other dedicated hardware, which was a project goal. llama.cpp gained traction with users lacking specialized hardware, as it could run on just a CPU. While initially designed for CPUs, GPU and NPU backend support was later added. As of May 2026, it has more than 109,000 stars on GitHub.

On April 30, 2024, FlashAttention was introduced.

On April 10, 2025, libmtmd was introduced, which reinvigorated support for multimodal models that had previously been stagnant.

On December 17, 2025, full acceleration on Android and ChromeOS devices was introduced via a new GUI binding, which unlocks native app development beyond the previous approach of cross-compiling and running CLI in an adb shell.

Architecture

llama.cpp supports multiple hardware targets, including x86, ARM, Metal, BLAS, BLIS, zDNN, ZenDNN, SYCL, MUSA, CUDA, HIP, CANN, OpenCL, RPC and Vulkan (version 1.2 or greater). These back-ends make up the GGML tensor library, used by the front-end model-specific llama.cpp code. llama.cpp makes use of several CPU extensions for optimization:

AVX, AVX2, AVX-512, AVX-VNNI and AMX for X86-64.

Neon, i8MM, SVE, SVE2, SME and SME2 for AArch64 (ARM64).

VXE2 (Vector Enhancement Facility 2) for S390x.

Apple silicon is an important target for the project.

llama.cpp supports a variety of features aimed at inference on edge devices, such as:

Ahead of time model quantization and on-the-fly kv-cache quantization.

Speculative decoding.

Partial offloading of model layers to system RAM, allowing devices to load models that would be too large to fit solely in GPU VRAM.

In addition, llama.cpp supports a variety of features and APIs for frontend communication, such as:

OpenAI-compatible endpoints like v1/chat/completions.

Grammar-based output formatting as JSON.

GGUF file format

The GGUF (GGML Universal File) file format is a binary format that stores both tensors and metadata in a single file, and is designed for fast saving, and loading of model data. It was introduced in August 2023 by the llama.cpp project to better maintain backwards compatibility as support was added for other model architectures. It superseded previous formats used by the project such as GGML and is typically produced by converting models developed with a different machine learning library such as PyTorch.

Design

GGUF focuses on quantization, the act of reducing precision in the model weights. This can lead to reduced memory usage and increased speed, albeit at the cost of reduced model accuracy.

GGUF supports 2-bit to 8-bit quantized integer types, common floating-point data formats such as float32, float16, and bfloat16, and 1.58-bit quantization.

GGUF contains information necessary for running a GPT-like language model such as the tokenizer vocabulary, context length, tensor info and other attributes.

Byte-level structure (little-endian)

Metadata block

Tensors info block

Models

Llama.cpp supports many large language models, including Llama, Mistral, Gemma, DeepSeek, gpt-oss, Phi and Qwen.

See also

LM Studio — desktop application for locally running and interacting with LLMs

Ollama

SGLang – framework for structured generation and high-performance large language model inference and serving

vLLM – large language model inference and serving engine

== References ==